

STensor/xTensorCustom

Setup

```
In[*]:= << mGRG`STensor`

In[*]:= $PreRead = ReplaceAll[#,
    expr_String => StringReplace[expr, "SP" -> "mGRG`STensor`Private"]] &;

In[*]:= DefKind[Greek, Alphabet["Greek"]]; DefKind[Capital, ToUpperCase /@ Alphabet[]]

In[*]:= Tdefine[T, "*"]; Tdefine[f[]]; Tdefine[v, 1];

In[*]:= Tdefine[B, 2]; Tdefine[S, "ba"]; Tdefine[A, "-ba"]; Tdefine[F, "-ba"]

In[*]:= DefDerivativeOperator[CovD, "D"]
```

flattenObject

Local functions for flattenObject

metricState

S가 대칭, A가 반대칭일 때, $S^{ab} \partial_c A_{ab}$ 와 $S^a_b \nabla_c A_a^b$ 는 0이지만 $S^a_b \partial_c A_a^b$ 는 0이 아니다. 이 때 더미 인덱스 각각의 metricState가 결정적 역할을 한다.

metricState는 인덱스의 현재 상태 정보를 돌려 준다. 첫 번째 요소는 dn/up, 두 번째 요소는 해당하는 인덱스 계열의 계량 텐서 유무이다. 마지막 요소는 moveFlag으로 계량 텐서가 없으면 아무런 역할을 하지 않고, 계량 텐서가 있으면 0/+1/-1 값을 갖는다. 여기서 그 값이 0이면 계량 텐서로 up/dn은 가능하지만 비공변 도함수의 작용 때문에 계량 텐서가 도함수 밖으로 이동할 수 없는 경우이고, +1/-1인 경우는 계량 텐서가 그에 따른 대칭 부호를 가지고 공변 도함수만 작용한 경우로 계량 텐서의 위치를 도함수 밖으로 이동 가능함을 의미한다.

Kdelta는 상수 텐서로 moveFlag은 0이다:

```
In[*]:= SP`metricState[la, Kdelta, {}]
Out[*]:= {-1, Metricg, 0}
```

BD는 비공변 도함수이므로 moveFlag은 0이다:

```
In[*]:= SP`metricState[la, BD, {}]
Out[*]:= {-1, Metricg, 0}
```

CovDs 옵션을 이용하여 BD를 공변 도함수로 취급할 수 있다:

```
In[*]:= SP`metricState[la, BD, {CovDs → {BD}}]
Out[*]= {-1, Metricg, 1}
```

IndexedObject가 아니면 계량 텐서가 없는 것으로 간주한다:

```
In[*]:= SP`metricState[ua, Other, {}]
Out[*]= {1, Null, 0}
```

IndexedOperand인 경우 계량 텐서가 있을 때 moveFlag은 해당하는 계량 텐서의 metricSymmetry인 +1/-1이다:

```
In[*]:= SP`metricState[la, T, {}]
Out[*]= {-1, Metricg, 1}
```

성분 인덱스는 인덱스 쌍이 될 수 없으므로 IndexedObject가 아닌 것과 동일하다:

```
In[*]:= SP`metricState[-1, T, {}]
Out[*]= {-1, Null, 0}
```

```
In[*]:= SP`metricState[-1, Other, {}]
Out[*]= {-1, Null, 0}
```

```
In[*]:= SP`metricState[anyExpr, more]
Out[*]= {Null, Null, 0}
```

reCheckMetricStates

미분 연산자가 작용하여 metricState를 변경시킬 수 있다. 미분 연산자가 계량 텐서에 대해 공변 도함수이면 변화가 없지만, 비공변 도함수이면 계량 텐서가 없고 metricFlag을 0으로 변경시킨다.

미분 연산자가 공변 도함수인 경우:

```
In[*]:= SP`reCheckMetricStates[{CD, la},
  {A, B, C, {{D, Metricg, 0}, {F, Metricg, 1}, {G, Metricg, -1}}}, {}]
Out[*]= {A, B, C, {{D, Metricg, 0}, {F, Metricg, 1}, {G, Metricg, -1}}}
```

```
In[*]:= SP`reCheckMetricStates[{CD, -1},
  {A, B, C, {{D, Metricg, 0}, {F, Metricg, 1}, {G, Metricg, -1}}}, {}]

Out[*]= {A, B, C, {{D, Metricg, 0}, {F, Metricg, 1}, {G, Metricg, -1}}}
```

미분 연산자가 공변 도함수이지만 CD type 연산자의 인자인 인덱스의 계열과 계량 텐서의 계열이 서로 다르면 metricState들이 {_, Null, 0}로 변경된다:

```
In[*]:= SP`reCheckMetricStates[{CD, 1a},
  {A, B, C, {{D, Metricg, 0}, {F, Metricg, 1}, {G, Metricg, -1}}}, {}]

Out[*]= {A, B, C, {{D, Null, 0}, {F, Null, 0}, {G, Null, 0}}}
```

LD와 BD는 공변 도함수가 아니다:

```
In[*]:= SP`reCheckMetricStates[{LD, v},
  {A, B, C, {{D, Metricg, 0}, {F, Metricg, 1}, {G, Metricg, -1}}}, {}]

Out[*]= {A, B, C, {{D, Null, 0}, {F, Null, 0}, {G, Null, 0}}}
```

```
In[*]:= SP`reCheckMetricStates[{BD, 1a},
  {A, B, C, {{D, Metricg, 0}, {F, Metricg, 1}, {G, Metricg, -1}}}, {}]

Out[*]= {A, B, C, {{D, Null, 0}, {F, Null, 0}, {G, Null, 0}}}
```

CovDs 옵션을 조정하여 LD나 BD를 공변 도함수로 취급할 수가 있다:

```
In[*]:= SP`reCheckMetricStates[{LD, v},
  {A, B, C, {{D, Metricg, 0}, {F, Metricg, 1}, {G, Metricg, -1}}}, {CovDs -> {LD}}]

Out[*]= {A, B, C, {{D, Metricg, 0}, {F, Metricg, 1}, {G, Metricg, -1}}}
```

```
In[*]:= SP`reCheckMetricStates[{BD, 1a},
  {A, B, C, {{D, Metricg, 0}, {F, Metricg, 1}, {G, Metricg, -1}}}, {CovDs -> {BD}}]

Out[*]= {A, B, C, {{D, Metricg, 0}, {F, Metricg, 1}, {G, Metricg, -1}}}
```

flattenObject

IndexedObject를 아래 형태의 내부 표현으로 바꾼다:

```
xObject[obj, nameL, {indexL, freeL, dummyL, mStateL}]
```

CovDs에 대한 옵션을 지정할 수 있다. (주목: freeL은 dropPairs[indexL] 명령으로 얻으므로 변수 이름이 의미하는 바와 달리 성분 인덱스가 포함될 수 있다.) mStateL은 metricState 명령으로 얻는다.

IndexedOperand

스칼라나 인덱스가 없는 임의 rank 텐서에 대해

In[*]:= SP`flattenObject[T[], {}]

Out[*]:= xObject[T, {T}, {{}}, {}, {}, {}]

In[*]:= SP`flattenObject[v[], {}] (* Error: nRank[v]=== 1 != 0 *)

Out[*]:=

v

유한한 rank의 텐서에 대해

In[*]:= SP`flattenObject[A[-2, la], {}]

Out[*]:= xObject[A_{2a}, {A}, {{-2, la}, {-2, la}, {}, {{-1, Null, 0}, {-1, Metricg, 1}}}]

In[*]:= SP`flattenObject[Kdelta[ub, la], {}]

Out[*]:= xObject[δ^b_a , {Kdelta},
{{ub, la}, {ub, la}, {}, {{1, Metricg, 0}, {-1, Metricg, 0}}}]

IndexedOperator: CD-type

공변 도함수 CD의 인자인 인덱스가 free 인덱스인 경우:

In[*]:= SP`flattenObject[CD[la, T[ub, uc]], {}]

Out[*]:= xObject[$\nabla_a T^{bc}$, {CD, T}, {{la, ub, uc}, {la, ub, uc},
{}, {{-1, Metricg, 1}, {1, Metricg, 1}, {1, Metricg, 1}}}]

공변 도함수 CD의 인자인 인덱스가 피 연산자의 인덱스와 쌍을 이룬 경우:

In[*]:= SP`flattenObject[CD[la, T[ub, ua]], {}]

Out[*]:= xObject[$\nabla_a T^{ba}$, {CD, T}, {{la, ub, ua}, {ub}, {la, ua},
{{-1, Metricg, 1}, {1, Metricg, 1}, {1, Metricg, 1}}}]

공변 도함수 CD의 인자인 인덱스가 성분 인덱스인 경우:

In[*]:= SP`flattenObject[CD[-1, T[ub, ua]], {}]

Out[*]:= xObject[$\nabla_1 T^{ba}$, {CD, T}, {{-1, ub, ua}, {-1, ub, ua},
{}, {{-1, Null, 0}, {1, Metricg, 1}, {1, Metricg, 1}}}]

공변 도함수 CD의 인자인 인덱스들이 피 연산자의 인덱스들과 쌍을 이룬 경우:

```
In[*]:= SP`flattenObject[CD[lb, la, T[ub, ua]], {}]
Out[*]= xObject[ $\nabla_b \nabla_a T^{ba}$ , {CD, CD, T}, {{lb, la, ub, ua}, {}, {lb, ub, la, ua},
  {{-1, Metricg, 1}, {-1, Metricg, 1}, {1, Metricg, 1}, {1, Metricg, 1}}}]
```

공변 도함수가 아닌 BD의 인자인 인덱스가 피 연산자의 인덱스와 쌍을 이룬 경우:

```
In[*]:= SP`flattenObject[BD[la, T[ub, ua]], {}]
Out[*]= xObject[ $\partial_a T^{ba}$ , {BD, T}, {{la, ub, ua}, {ub}, {la, ua}, {{-1, Metricg, 0}, {1, Null, 0}, {1, Null, 0}}}]
```

CovDs 옵션을 조정하여 BD를 공변 도함수로 취급할 수 있다. BD의 인자인 인덱스가 피 연산자의 인덱스와 쌍을 이룬 경우:

```
In[*]:= SP`flattenObject[BD[la, T[ub, ua]], {CovDs -> {BD}}]
Out[*]= xObject[ $\partial_a T^{ba}$ , {BD, T}, {{la, ub, ua}, {ub}, {la, ua},
  {{-1, Metricg, 1}, {1, Metricg, 1}, {1, Metricg, 1}}}]
```

공변 도함수가 아닌 BD가 작용한 후에 공변 도함수인 CD가 작용한 경우. 이 때 도함수의 인자인 인덱스들이 피 연산자의 인덱스들과 쌍을 이룬다면:

```
In[*]:= SP`flattenObject[CD[lb, BD[la, T[ub, ua]]], {}]
Out[*]= xObject[ $\nabla_b \partial_a T^{ba}$ , {CD, BD, T}, {{lb, la, ub, ua}, {}, {lb, ub, la, ua},
  {{-1, Metricg, 1}, {-1, Metricg, 0}, {1, Null, 0}, {1, Null, 0}}}]
```

공변 도함수 CD의 인자인 인덱스들 중에서 한 개가 피 연산자의 인덱스와 쌍을 이루고, 나머지 하나는 성분 인덱스인 경우:

```
In[*]:= SP`flattenObject[CD[lb, -1, T[ub, ua]], {}]
Out[*]= xObject[ $\nabla_b \nabla_1 T^{ba}$ , {CD, CD, T}, {{lb, -1, ub, ua}, {-1, ua}, {lb, ub},
  {{-1, Metricg, 1}, {-1, Null, 0}, {1, Metricg, 1}, {1, Metricg, 1}}}]
```

IndexedOperator: LD-type

LD 연산자는 기본적으로 공변 연산자가 아니다:

```
In[*]:= SP`flattenObject[LD[v, T[ub, ua]], {}]
Out[*]= xObject[ $\mathcal{L}_v T^{ba}$ , {LD, v, T}, {{ub, ua}, {ub, ua}, {}, {{1, Null, 0}, {1, Null, 0}}}]
```

공변 연산자가 아닌 LD 연산자가 작용한 후에 공변 연산자 CD가 작용한 경우:

```
In[*]:= SP`flattenObject[CD[la, LD[v, T[ub, ua]]], {}]
```

```
Out[*]:= xObject[ $\nabla_a \mathcal{L}_v T^{ba}$ , {CD, LD, v, T},  
{ {la, ub, ua}, {ub}, {la, ua}, {{-1, Metricg, 1}, {1, Null, 0}, {1, Null, 0}}}]
```

CovDs 옵션을 조정하여 임시로 공변 연산자를 설정할 수 있다:

```
In[*]:= SP`flattenObject[CD[la, LD[v, T[ub, ua]]], {CovDs → {CD, LD}}]
```

```
Out[*]:= xObject[ $\nabla_a \mathcal{L}_v T^{ba}$ , {CD, LD, v, T}, {{la, ub, ua}, {ub},  
{la, ua}, {{-1, Metricg, 1}, {1, Metricg, 1}, {1, Metricg, 1}}}]
```

BD와 LD 연산자는 공변 연산자가 아니다:

```
In[*]:= SP`flattenObject[BD[la, LD[v, T[ub, ua]]], {}]
```

```
Out[*]:= xObject[ $\partial_a \mathcal{L}_v T^{ba}$ , {BD, LD, v, T},  
{ {la, ub, ua}, {ub}, {la, ua}, {{-1, Metricg, 0}, {1, Null, 0}, {1, Null, 0}}}]
```

IndexedOperator: XD-type

```
In[*]:= SP`defineOperator[Lap, " $\partial^2$ ", XD]
```

Lap 연산자는 공변 연산자가 아니다:

```
In[*]:= SP`flattenObject[Lap[T[ub, ua]], {}]
```

```
Out[*]:= xObject[ $\partial^2 T^{ba}$ , {Lap, T}, {{ub, ua}, {ub, ua}, {}, {{1, Null, 0}, {1, Null, 0}}}]
```

CovDs 옵션을 조정하여 임시로 Lap가 공변 연산자가 되도록 설정할 수 있다:

```
In[*]:= SP`flattenObject[Lap[T[ub, ua]], {CovDs → {CD, Lap}}]
```

```
Out[*]:= xObject[ $\partial^2 T^{ba}$ , {Lap, T},  
{ {ub, ua}, {ub, ua}, {}, {{1, Metricg, 1}, {1, Metricg, 1}}}]
```

IndexedOperator: XP-type

```
In[*]:= SP`defineOperator[oXP, "*", XP]
```

XP type 연산자가 포함된 IndexedObject에 대해서는 (현재까지는) 특별한 연산을 하지 않고, 단지 \$flattenOTHER로 둘러 싸기만 한다:

```
In[*]:= SP`flattenObject[oXP[T[ub, ua], A[lb, la], v[lc]], {}]
```

```
Out[*]:= mGRG`STensor`Private`$flattenOTHER[oXP[Tba, Aba, vc]]
```

Scalar-functions:

IndexedObject의 스칼라 표현에서 모든 인덱스들은 쌍으로 존재해야 한다.

(xSort 함수의 호출에 의한) scalarCan 함수는 인자를 'Tsimply'로 단순화하고, 다시 ResetDummies 함수를 적용한다.

Tscalar로 둘러 싸인 스칼라 표현의 경우 scalarCan 함수를 적용하여 인덱스 쌍을 정규 인덱스로 만든다:

```
In[*]:= SP`flattenObject[Tscalar[A[uc, ud] × A[ld, lc]], {}]
```

```
Out[*]= xObject[-(Aab Aab), {-Aab Aab}, {{}, {}, {}, {}}]
```

Log 함수와 같이 인자가 한 개인 스칼라 함수에 대해서는 인자를 Tscalar로 둘러싼 후 flattenObject 함수를 적용한 결과를 다시 원래의 스칼라 함수로 둘러싼다:

```
In[*]:= SP`flattenObject[Log[A[ud, uc] × A[ld, lc]], {}]
```

```
Out[*]= xObject[Log[(Aab Aab)], {Log, Aab Aab}, {{}, {}, {}, {}}]
```

Power 함수와 같이 인자가 두 개인 스칼라 함수에 대해서는 각각의 인자를 Tscalar로 둘러싼 후 scalarCan 함수를 적용하고, 그 결과를 다시 원래의 스칼라 함수로 둘러싼다:

```
In[*]:= SP`flattenObject[Tscalar[A[ud, uc] × A[lc, ld]]2, {}]
```

```
Out[*]= xObject[(Aab Aba)2, {Power, (Aab Aba), 2}, {{}, {}, {}, {}}]
```

```
In[*]:= SP`flattenObject[Tscalar[A[ud, ua] × A[ld, la]]v[lc] × v[uc], {}]
```

```
Out[*]= xObject[(Aab Aab)(va va), {Power, (Aab Aab), (va va)}, {{}, {}, {}, {}}]
```

Plus

flattenObject 함수는 단일 항을 위한 함수이므로 다중 항으로 구성된 Plus 표현은 \$flattenOTHER로 둘러싸서 더 이상 단순화 연산이 동작하지 않도록 한다:

```
In[*]:= SP`flattenObject[T[] × A[lb, la] + B[la, lb], {}]
```

```
Out[*]= mGRG`STensor`Private`$flattenOTHER[Bab + Aba T]
```

Times

IndexedObject들의 곱으로 구성된 표현은 xTensorTimes를 Head로 갖는 표현으로 변경한 후, 각각의 Object에 flattenObject 함수를 적용한다:

```
In[*]:= SP`flattenObject[T[] × A[lb, la], {}]
```

```
Out[*]=
```

```
xTensorTimes[xObject[Aba, {A},
  {{lb, la}, {lb, la}, {}, {{-1, Metricg, 1}, {-1, Metricg, 1}}}],
xObject[T, {T}, {{}, {}, {}, {}]]]
```

```
In[*]:= SP`flattenObject[T[] × A[lb, la] × B[la, lb], {}]
```

```
Out[*]=
```

```
xTensorTimes[xObject[Aba, {A},
  {{lb, la}, {lb, la}, {}, {{-1, Metricg, 1}, {-1, Metricg, 1}}}], xObject[
  Bab, {B}, {{la, lb}, {la, lb}, {}, {{-1, Metricg, 1}, {-1, Metricg, 1}}}],
xObject[T, {T}, {{}, {}, {}, {}]]]
```

Others

errObject와 \$flattenOTHER를 Head로 갖는 표현은 그대로 놔둔다:

```
In[*]:= SP`flattenObject[SP`errObject[something], {}]
```

```
Out[*]=
```

```
something
```

```
In[*]:= % // FullForm
```

```
Out[*] // FullForm =
```

```
mGRG`STensor`Private`errObject[something]
```

참고: errObject는 MakeBoxes를 이용하여 출력 형태를 지정하였다:

```
In[*]:= ? SP`errObject
```

```
Out[*]=
```

```
Symbol

mGRG`STensor`Private`errObject

Format Definitions

mGRG`STensor`Private`errObject /: MakeBoxes[mGRG`STensor`Private`errObject[m
Full Name  mGRG`STensor`Private`errObject
^
```

```
In[*]:= SP`flattenObject[SP`$flattenOTHER[something], {}]
```

```
Out[*]=
```

```
mGRG`STensor`Private`$flattenOTHER[something]
```

처리 방법을 정하지 않은 나머지 모든 표현은 \$flattenOTHER로 돌려 쓴다:


```
In[*]:= SP`flattenObject[nonT[la, lb], {}]
Out[*]= mGRG`STensor`Private`$flattenOTHER[nonT[la, lb]]
```

check

입력한 표현에 오류가 발견되면 flattenObject는 그 표현의 Head를 ErrorT로 돌려 싸고, 전체 표현을 errObject로 돌려 쏜다.

텐서의 rank와 입력한 인덱스의 수가 동일해야 한다. 또한 인덱스 계열도 일치해야 한다:

```
In[*]:= SP`flattenObject[#, {}] & /@ {A[], A[la], A[la, lb], A[la, lb]}
Out[*]= {A, Aa, AAb, xObject[Aab, {A},
  {{la, lb}, {la, lb}, {}, {{-1, Metricg, 1}, {-1, Metricg, 1}}}]}
```

CD type 연산자의 첫 번째 인자는 인덱스가 갖추어야 할 특성을 유지해야 한다:

```
In[*]:= SP`flattenObject[#, {}] & /@
  {CD[la, T[ub, uc]], CD[la, T[ub, la]], CD[la, T[ub, ua]]}
Out[*]= { $\nabla_A T^{bc}$ ,  $\nabla_a T^b_a$ , xObject[ $\nabla_a T^{ba}$ , {CD, T}, {{la, ub, ua}, {ub},
  {la, ua}, {{-1, Metricg, 1}, {1, Metricg, 1}, {1, Metricg, 1}}}]}
```

XP type 연산자에 대해서 flattenObject는 인자들에 있는 인덱스의 중복을 확인한 후, 문제가 없으면 전체 표현을 \$flattenOTHER로 돌려 쏜다:

```
In[*]:= SP`flattenObject[#, {}] & /@
  {oXP[T[ub, ua], A[lb, la], v[la]], oXP[T[ub, ua], A[lb, lc], v[la]]}
Out[*]= {oXP[Tba, Aba, va], mGRG`STensor`Private`$flattenOTHER[oXP[Tba, Abc, va]]}
```

텐서 곱 표현은 각각의 Object에 대해 독립적으로 flattenObject 함수를 적용하므로 텐서 곱 전체의 중복된 인덱스에 대한 오류는 (flattenObject 함수가) 찾지 못한다. 아래 표현은 인덱스 lb가 중복된 잘못된 텐서 곱이다:

```
In[*]:= SP`flattenObject[T[lb] × A[lb, la], {}]
Out[*]= xTensorTimes[xObject[Aba, {A},
  {{lb, la}, {lb, la}, {}, {{-1, Metricg, 1}, {-1, Metricg, 1}}}],
  xObject[Tb, {T}, {{lb}, {lb}, {}, {{-1, Metricg, 1}}}]
```

스칼라 표현에는 free 인덱스가 없어야 한다:

```

In[*]:= SP`flattenObject[#, {}] & /@ {Tscalar[A[ub, ua] × A[lb, ua]],
      Log[A[ub, ua] × A[lb, ua]], (A[ub, ua] × A[lb, la])2,
      Tscalar[A[ub, ua] × A[lb, ua]]2, Tscalar[A[ub, ua] × A[lb, la]]v[la] × v[la]}

Out[*]= { (Aba Aba), Log[Aba Aba], xTensorTimes[Aba2, Aba2], (Aba Aba)2, (Aba Aba)va2 }

```

xSort

Local functions for xSort

markNoIndex

Object의 인덱스가 없으면 그 object를 xNoIndex로 돌려 쓴다.

명백히 인덱스가 없는 경우:

```

In[*]:= SP`markNoIndex[SP`flattenObject[T[], {}]]

Out[*]= xNoIndex[xObject[T, {T}, {{}, {}, {}, {}]]

```

\$flattenOTHER로 돌려 싸인 표현은 인덱스가 없는 것으로 간주한다:

```

In[*]:= SP`flattenObject[oXP[T[ub, ua], A[lb, la], v[lc]], {}]
      SP`markNoIndex[%]

Out[*]= mGRG`STensor`Private`$flattenOTHER[oXP[Tba, Aba, vc]]

Out[*]= xNoIndex[oXP[Tba, Aba, vc]]

```

xTensorTimes으로 돌려 싸인 표현은 각각의 object에 독립적으로 작용한다:

```
In[*]:= SP`flattenObject[T[ua] × T[] × T[-1] × S[ud, uf], {}]
SP`markNoIndex[%]
```

```
Out[*]=
```

```
xTensorTimes[
  xObject[Sdf, {S}, {{ud, uf}, {ud, uf}, {}, {{1, Metricg, 1}, {1, Metricg, 1}}}],
  xObject[T, {T}, {{}, {}, {}, {}]],
  xObject[T1, {T}, {{-1}, {-1}, {}, {{-1, Null, 0}}}],
  xObject[Ta, {T}, {{ua}, {ua}, {}, {{1, Metricg, 1}}}] ]
```

```
Out[*]=
```

```
xTensorTimes[
  xObject[Sdf, {S}, {{ud, uf}, {ud, uf}, {}, {{1, Metricg, 1}, {1, Metricg, 1}}}],
  xNoIndex[xObject[T, {T}, {{}, {}, {}, {}]]],
  xObject[T1, {T}, {{-1}, {-1}, {}, {{-1, Null, 0}}}],
  xObject[Ta, {T}, {{ua}, {ua}, {}, {{1, Metricg, 1}}}] ]
```

freeIndicesOf

Object의 free 인덱스를 돌려 준다:

```
In[*]:= SP`flattenObject[T[-2, la, lb, ua], {}]
SP`freeIndicesOf[%]
```

```
Out[*]=
```

```
xObject[T2aba, {T}, {{-2, la, lb, ua}, {-2, lb}, {la, ua},
  {{-1, Null, 0}, {-1, Metricg, 1}, {-1, Metricg, 1}, {1, Metricg, 1}}}]
```

```
Out[*]=
```

```
{-2, lb}
```

Head가 xNoIndex, errObject, \$flattenOTHER이면 free 인덱스는 없다:

```
In[*]:= xNoIndex[expr1] // SP`freeIndicesOf
SP`errObject[expr2] // SP`freeIndicesOf
SP`$flattenOTHER[expr3] // SP`freeIndicesOf
```

```
Out[*]=
```

```
{}
```

```
Out[*]=
```

```
{}
```

```
Out[*]=
```

```
{}
```

Head가 xCommutingObjects이거나 xTensorTimes이면 각각의 object에 대해 freeIndicesOf를 적용하여 free 인덱스들을 얻고, 그 인덱스들을 모두 합한 후 dropPairs 함수를 이용하여 free 인덱스를 얻는다:

```
In[*]:= expr = SP`flattenObject[T[ub, la] × A[lb, -1], {}]
```

```
Out[*]:= xTensorTimes[xObject[Ab1, {A},
  {{lb, -1}, {lb, -1}, {}, {{-1, Metricg, 1}, {-1, Null, 0}}}], xObject[
  Tba, {T}, {{ub, la}, {ub, la}, {}, {{1, Metricg, 1}, {-1, Metricg, 1}}]]
```

```
In[*]:= SP`freeIndicesOf /@ expr
```

```
Out[*]:= xTensorTimes[{lb, -1}, {ub, la}]
```

```
In[*]:= Join@@List@@%
```

```
Out[*]:= {lb, -1, ub, la}
```

```
In[*]:= % // SP`dropPairs (* 성분 인덱스를 포함함을 주목 *)
```

```
Out[*]:= {-1, la}
```

```
In[*]:= % == SP`freeIndicesOf[expr]
```

```
Out[*]:= True
```

mStateOrderItems

objectOrder 함수를 위해 계량 텐서가 없으면 up/dn 정보를 유지하지만 계량 텐서가 있으면 up/dn 정보가 의미가 없도록 한다.

```
In[*]:= SP`mStateOrderItems[{dnup, Null, _}] (*keep dn/up*)
```

```
Out[*]:= dnup
```

```
In[*]:= SP`mStateOrderItems[{dnup, metric, _}] (*neglect dn/up*)
```

```
Out[*]:= metric
```

objectOrderItems

objectOrder 함수에서의 정렬을 위해 아이템들을 재구성한다.

```
In[*]:= expr = SP`flattenObject[T[ub, la] × A[lb, lc], {}]
```

```
Out[*]:= xTensorTimes[xObject[Abc, {A},  
  {{lb, lc}, {lb, lc}, {}, {{-1, Metricg, 1}, {-1, Metricg, 1}}}], xObject[  
  Tba, {T}, {{ub, la}, {ub, la}, {}, {{1, Metricg, 1}, {-1, Metricg, 1}}]]
```

```
In[*]:= {o1, nameL1, {idxL1, free1, dum1, ms1}} = List @@ expr[[1];  
{o2, nameL2, {idxL2, free2, dum2, ms2}} = List @@ expr[[2];
```

```
In[*]:= SP`$freesForSorting = SP`freeIndicesOf[expr]
```

```
Out[*]:= {lc, la}
```

```
In[*]:= frees1 = Intersection[free1, SP`$freesForSorting];  
frees2 = Intersection[free2, SP`$freesForSorting];  
{frees1, frees2}
```

```
Out[*]:= {{lc}, {la}}
```

```
In[*]:= {nameL1, Length[idxL1], -Length[frees1], SP`mStateOrderItems /@ ms1}
```

```
Out[*]:= {{A}, 2, -1, {Metricg, Metricg}}
```

```
In[*]:= % == SP`objectOrderItems[expr[[1]]]
```

```
Out[*]:= True
```

```
In[*]:= {nameL2, Length[idxL2], -Length[frees2], SP`mStateOrderItems /@ ms2}
```

```
Out[*]:= {{T}, 2, -1, {Metricg, Metricg}}
```

```
In[*]:= % == SP`objectOrderItems[expr[[2]]]
```

```
Out[*]:= True
```

objectOrderQ

첫 번째 인자의 Head가 xNoIndex이면 무조건 True이고, 두 번째 인자의 Head가 xNoIndex이면 False이다:

```
In[*]:= SP`objectOrderQ[xNoIndex[expr1], expr2]
SP`objectOrderQ[xNoIndex[expr1], xNoIndex[expr2]]
SP`objectOrderQ[expr1, xNoIndex[expr2]]
```

```
Out[*]:= True
```

```
Out[*]:= True
```

```
Out[*]:= False
```

두 object 사이의 objectOrderQ (objectOrderEqualQ)는 objectOrderItems 함수를 각각의 인자에 작용한 후 Order 명령의 결과값이 ≥ 0 이면 True이다:

```
In[*]:= expr = SP`flattenObject[T[ub, la]  $\times$  A[lb, lc], {}]
```

```
Out[*]:= xTensorTimes[xObject[Abc, {A},
  {{lb, lc}, {lb, lc}, {}, {{-1, Metricg, 1}, {-1, Metricg, 1}}}], xObject[
  Tab, {T}, {{ub, la}, {ub, la}, {}, {{1, Metricg, 1}, {-1, Metricg, 1}}]]
```

```
In[*]:= SP`objectOrderItems /@ {expr[[1]], expr[[2]]}
```

```
Out[*]:= {{{A}, 2, -1, {Metricg, Metricg}}, {{T}, 2, -1, {Metricg, Metricg}}}
```

```
In[*]:= Order @@ %
```

```
Out[*]:= 1
```

```
In[*]:= (%  $\geq 0$ ) === SP`objectOrderQ[expr[[1]], expr[[2]]]
```

```
Out[*]:= True
```

```
In[*]:= SP`objectOrderItems /@ {expr[[1]], expr[[1]]}
```

```
Out[*]:= {{{A}, 2, -1, {Metricg, Metricg}}, {{A}, 2, -1, {Metricg, Metricg}}}
```

```
In[*]:= Order @@ %
```

```
Out[*]:= 0
```

```
In[*]:= (% === 0) === SP`objectOrderQ[expr[[1]], expr[[1]]]
```

```
Out[*]:= True
```

breakCommutings

In[*]:= SP`breakCommutings[xTensorTimes[x]]

Out[*]=

x

In[*]:= SP`breakCommutings[xTensorTimes[x, x]]

Out[*]=

xCommutingObjects[x, x]

objectSort

Head가 xTensorTimes가 아니면 인자를 그대로 돌려준다:

In[*]:= SP`objectSort[xObject[some]]

SP`objectSort[xNoIndex[some]]

SP`objectSort[SP`errObject[some]]

SP`objectSort[SP`\$flattenOTHER[some]]

Out[*]=

xObject[some]

Out[*]=

xNoIndex[some]

Out[*]=

some

Out[*]=

mGRG`STensor`Private`\$flattenOTHER[some]

인자가 xTensorTimes이면 freeIndicesOf 함수로 free 인덱스를 얻어서 \$freesForSorting 변수에 저장한다. 그리고 objectOrderQ를 이용하여 정렬한 후, objectOrderQ 기준으로 동일한 object 표현끼리 Split 명령으로 잘라낸다. 끝으로 잘라낸 각각의 표현에 breakCommutings 명령을 적용하여 두 개 이상의 object를 가진 조각을 xCommutingObjects로 돌려 쏜다.

In[*]:= expr = SP`flattenObject[T[-1] × T[-2], {}]

Out[*]=

xTensorTimes[xObject[T₂, {T}, {{-2}, {-2}, {}, {{-1, Null, 0}}}],
xObject[T₁, {T}, {{-1}, {-1}, {}, {{-1, Null, 0}}]]

In[*]:= SP`\$freesForSorting = SP`freeIndicesOf[expr]

Out[*]=

{-2, -1}

In[*]:= **Sort**[expr, SP`objectOrderQ]

Out[*]=
 $\text{xTensorTimes}[\text{xObject}[T_2, \{T\}, \{\{-2\}, \{-2\}, \{\}, \{-1, \text{Null}, 0\}\}],$
 $\text{xObject}[T_1, \{T\}, \{\{-1\}, \{-1\}, \{\}, \{-1, \text{Null}, 0\}\}]]$

In[*]:= **Split**[% , SP`objectOrderEqualQ]

Out[*]=
 $\text{xTensorTimes}[\text{xTensorTimes}[\text{xObject}[T_2, \{T\}, \{\{-2\}, \{-2\}, \{\}, \{-1, \text{Null}, 0\}\}],$
 $\text{xObject}[T_1, \{T\}, \{\{-1\}, \{-1\}, \{\}, \{-1, \text{Null}, 0\}\}]]]$

In[*]:= **SP`breakCommutings** /@ %

Out[*]=
 $\text{xTensorTimes}[$
 $\text{xCommutingObjects}[\text{xObject}[T_2, \{T\}, \{\{-2\}, \{-2\}, \{\}, \{-1, \text{Null}, 0\}\}],$
 $\text{xObject}[T_1, \{T\}, \{\{-1\}, \{-1\}, \{\}, \{-1, \text{Null}, 0\}\}]]]$

In[*]:= **% == SP`objectSort**[expr]

Out[*]=
 True

In[*]:= **SP`objectSort**[SP`flattenObject[T[1] × T[2], {}]]

Out[*]=
 $\text{xTensorTimes}[$
 $\text{xCommutingObjects}[\text{xObject}[T^1, \{T\}, \{\{1\}, \{1\}, \{\}, \{1, \text{Null}, 0\}\}],$
 $\text{xObject}[T^2, \{T\}, \{\{2\}, \{2\}, \{\}, \{1, \text{Null}, 0\}\}]]]$

In[*]:= **SP`objectSort**[SP`flattenObject[T[-2] × T[2], {}]]

Out[*]=
 $\text{xTensorTimes}[\text{xObject}[T_2, \{T\}, \{\{-2\}, \{-2\}, \{\}, \{-1, \text{Null}, 0\}\}],$
 $\text{xObject}[T^2, \{T\}, \{\{2\}, \{2\}, \{\}, \{1, \text{Null}, 0\}\}]]]$

xSort and xUnSort

xSort는 flattenObject, markNoIndex, objectSort 함수들을 순서대로 적용하고, 결과적으로 텐서 곱 표현에서 xCommutingObjects를 찾아 낸다.

xUnSort는 xSort를 취소한다.

In[*]:= SP`xSort[v[ua]]
SP`xUnSort[%]

Out[*]=

xObject[v^a, {v}, {{ua}, {ua}}, {}, {{1, Metricg, 1}}]

Out[*]=

v^a

In[*]:= SP`xSort[v[ua] × v[ub]]
SP`xUnSort[%]

Out[*]=

xTensorTimes[
xCommutingObjects[xObject[v^a, {v}, {{ua}, {ua}}, {}, {{1, Metricg, 1}}],
xObject[v^b, {v}, {{ub}, {ub}}, {}, {{1, Metricg, 1}}]]]

Out[*]=

v^a v^b

In[*]:= SP`xSort[T[ua, ub, uc] × v[lb] × v[lc]]
SP`xUnSort[%]

Out[*]=

xTensorTimes[xObject[T^{abc}, {T}, {{ua, ub, uc}, {ua, ub, uc}},
{}, {{1, Metricg, 1}, {1, Metricg, 1}, {1, Metricg, 1}}],
xCommutingObjects[xObject[v_b, {v}, {{lb}, {lb}}, {}, {{-1, Metricg, 1}}],
xObject[v_c, {v}, {{lc}, {lc}}, {}, {{-1, Metricg, 1}}]]]

Out[*]=

T^{abc} v_b v_c

In[*]:= SP`xSort[T[ua, ub, uc] × T[ud, lb, lc]]
SP`xUnSort[%]

Out[*]=

xTensorTimes[
xCommutingObjects[xObject[T^{abc}, {T}, {{ua, ub, uc}, {ua, ub, uc}}, {},
{{1, Metricg, 1}, {1, Metricg, 1}, {1, Metricg, 1}}],
xObject[T^d_{bc}, {T}, {{ud, lb, lc}, {ud, lb, lc}}, {},
{{1, Metricg, 1}, {-1, Metricg, 1}, {-1, Metricg, 1}}]]]

Out[*]=

T^{abc} T^d_{bc}

See mStateOrderItems[]: 계량 텐서가 없으면 dn/up 상태를 바꾸지 못한다.

In[*]:= Off[MetricgFlag]

계량 텐서가 없으면 v^a는 v_a로부터 얻을 수 없다:

In[*]:= SP`xSort[v[la] × v[lb]]

Out[*]=

```
xTensorTimes[
  xCommutingObjects[xObject[va, {v}], {{la}, {la}}, {}, {{-1, Null, 1}}]],
  xObject[vb, {v}], {{lb}, {lb}}, {}, {{-1, Null, 1}}]]]
```

In[*]:= SP`xSort[v[ua] × v[lb]]

Out[*]=

```
xTensorTimes[xObject[vb, {v}], {{lb}, {lb}}, {}, {{-1, Null, 1}}], va]
```

사실, 계량 텐서가 없으므로 v^a 는 잘못된 표현이다:

In[*]:= SP`checkObject[v, {ua}]

Out[*]=

False

In[*]:= SP`flattenObject[v[ua], {}]

Out[*]=

v^a

계량 텐서가 있으면 dn/up 상태를 바꾸는 것이 가능하다:

In[*]:= On[MetricgFlag]

In[*]:= SP`xSort[v[la] × v[ub]]

Out[*]=

```
xTensorTimes[
  xCommutingObjects[xObject[va, {v}], {{la}, {la}}, {}, {{-1, Metricg, 1}}]],
  xObject[vb, {v}], {{ub}, {ub}}, {}, {{1, Metricg, 1}}]]]
```

In[*]:= SP`xSort[BD[ua, f[]] × BD[lb, f[]]]

Out[*]=

```
xTensorTimes[
  xCommutingObjects[xObject[∂bf, {BD, f}], {{lb}, {lb}}, {}, {{-1, Metricg, 0}}]],
  xObject[∂af, {BD, f}], {{ua}, {ua}}, {}, {{1, Metricg, 0}}]]]
```

BD는 공변 도함수가 아니다:

In[*]:= SP`xSort[BD[ua, T[ub, uc]] × BD[la, T[lb, lc]]]

Out[*]=

```
xTensorTimes[xObject[∂aTbc, {BD, T}], {{la, lb, lc},
  {la, lb, lc}}, {}, {{-1, Metricg, 0}, {-1, Null, 0}, {-1, Null, 0}}]],
  xObject[∂aTbc, {BD, T}], {{ua, ub, uc}, {ua, ub, uc}}, {},
  {{1, Metricg, 0}, {1, Null, 0}, {1, Null, 0}}]]]
```

CovDs 옵션을 조정하여 BD를 공변 도함수로 취급할 수가 있다:

```
In[*]:= SP`xSort[BD[ua, T[ub, uc]] × BD[la, T[lb, lc]], CovDs → {CD, BD}]
Out[*]=
xTensorTimes[
  xCommutingObjects[xObject[ $\partial_a T_{bc}$ , {BD, T}, {{la, lb, lc}, {la, lb, lc},
    {}}, {{-1, Metricg, 1}, {-1, Metricg, 1}, {-1, Metricg, 1}}]],
  xObject[ $\partial^a T^{bc}$ , {BD, T}, {{ua, ub, uc}, {ua, ub, uc}, {}},
    {{1, Metricg, 1}, {1, Metricg, 1}, {1, Metricg, 1}}]]]
```

공변 도함수가 아닌 다른 연산자도 BD와 유사하게 연산된다:

```
In[*]:= SP`defineOperator[someD, "D", XD]
In[*]:= SP`xSort[someD[T[ua, ub, uc]] × CD[la, T[lb, lc]]]
Out[*]=
xTensorTimes[xObject[ $\nabla_a T_{bc}$ , {CD, T}, {{la, lb, lc}, {la, lb, lc},
  {}}, {{-1, Metricg, 1}, {-1, Metricg, 1}, {-1, Metricg, 1}}]],
  xObject[D $T^{abc}$ , {someD, T}, {{ua, ub, uc}, {ua, ub, uc},
    {}}, {{1, Null, 0}, {1, Null, 0}, {1, Null, 0}}]]]
```

```
In[*]:= SP`xSort[LD[v, T[ub, lc]] × LD[v, T[lb, uc]]]
Out[*]=
xTensorTimes[xObject[ $\mathcal{L}_v T_b^c$ , {LD, v, T},
  {{lb, uc}, {lb, uc}, {}}, {{-1, Null, 0}, {1, Null, 0}}]], xObject[ $\mathcal{L}_v T_c^b$ ,
  {LD, v, T}, {{ub, lc}, {ub, lc}, {}}, {{1, Null, 0}, {-1, Null, 0}}]]]
```

```
In[*]:= SP`xSort[LD[v, T[ub, lc]] × LD[v, T[lb, ua]]]
Out[*]=
xTensorTimes[xObject[ $\mathcal{L}_v T_b^a$ , {LD, v, T},
  {{lb, ua}, {lb, ua}, {}}, {{-1, Null, 0}, {1, Null, 0}}]], xObject[ $\mathcal{L}_v T_c^b$ ,
  {LD, v, T}, {{ub, lc}, {ub, lc}, {}}, {{1, Null, 0}, {-1, Null, 0}}]]]
```

```
In[*]:= SP`xSort[LD[v, T[ub, lc]] × LD[v, T[lb, ua]], CovDs → {CD, LD}]
Out[*]=
xTensorTimes[
  xCommutingObjects[xObject[ $\mathcal{L}_v T_b^a$ , {LD, v, T}, {{lb, ua}, {lb, ua}, {}},
    {{-1, Metricg, 1}, {1, Metricg, 1}}]], xObject[ $\mathcal{L}_v T_c^b$ , {LD, v, T},
    {{ub, lc}, {ub, lc}, {}}, {{1, Metricg, 1}, {-1, Metricg, 1}}]]]
```

errObject:

In[*]:= SP`xSort[Tscalar[T[la, lb]]]

Out[*]=

(T_{ab})

In[*]:= SP`xSort[Log[T[la, lb]]]

Out[*]=

$\text{Log}[T_{ab}]$

In[*]:= SP`xSort /@ {A[], A[la], A[la, lb], A[la, lb]}

SP`xUnSort /@ %

Out[*]=

$\{A, A_a, A_{Ab}, \text{xObject}[A_{ab}, \{A\}, \{\{la, lb\}, \{la, lb\}, \{\}, \{\{-1, \text{Metricg}, 1\}, \{-1, \text{Metricg}, 1\}\}\}]\}$

Out[*]=

$\{A, A_a, A_{Ab}, A_{ab}\}$

In[*]:= SP`xSort /@ {CD[la, T[ub, uc]], CD[la, T[ub, la]], CD[la, T[ub, ua]]}

SP`xUnSort /@ %

Out[*]=

$\{\nabla_A T^{bc}, \nabla_a T^b_a, \text{xObject}[\nabla_a T^{ba}, \{CD, T\}, \{\{la, ub, ua\}, \{ub\}, \{la, ua\}, \{\{-1, \text{Metricg}, 1\}, \{1, \text{Metricg}, 1\}, \{1, \text{Metricg}, 1\}\}\}]\}$

Out[*]=

$\{\nabla_A T^{bc}, \nabla_a T^b_a, \nabla_a T^{ba}\}$

Any others:

In[*]:= SP`xSort[a]

% // SP`xUnSort

Out[*]=

xNoIndex[a]

Out[*]=

a

metricStatesOf and xSymmetryOf

metricStatesOf

Extract metricStates from xSorted objects

```

In[*]:= SP`xSort[A[-2, la], {}]
SP`metricStatesOf[%]

Out[*]=
xObject[A2a, {A}, {{-2, la}, {-2, la}, {}}, {{-1, Null, 0}, {-1, Metricg, 1}}]

Out[*]=
{{-1, Null, 0}, {-1, Metricg, 1}}

In[*]:= SP`xSort[T[], {}]
SP`metricStatesOf[%]

Out[*]=
xNoIndex[xObject[T, {T}, {{}}, {}], {}]

Out[*]=
{}

In[*]:= SP`xSort[v[la] × v[lb], {}]
SP`metricStatesOf[%]

Out[*]=
xTensorTimes[
  xCommutingObjects[xObject[va, {v}, {{la}, {la}, {}}, {{-1, Metricg, 1}}],
  xObject[vb, {v}, {{lb}, {lb}, {}}, {{-1, Metricg, 1}}]]

Out[*]=
{{-1, Metricg, 1}, {-1, Metricg, 1}}

```

Local functions for xSymmetryOf

```

xSymmetry[
  the number of slots,
  expr with all indices replaced by slot numbers,
  a list of rules xSlot[n] -> index,
  a Generating Set of expr
]

```

xSlot

인덱스의 슬롯 표기:

```

In[*]:= {xSlot[1], xSlot[3], xSlot[5]}

Out[*]=
{●1, ●3, ●5}

```

toSlotRules

첫 번째 인자 만큼의 슬롯이 이미 있을 때 슬롯 번호와 두 번째 인자인 인덱스들을 연결하는 규칙을 생성한다.

```
In[*]:= SP`toSlotRules[3, {ua, lb, c_, _, 1, -2}]
Out[*]:= {
  ●4 → ua, ●5 → lb, ●6 → c_, ●7 → _, ●8 → 1, ●9 → -2
}
```

```
In[*]:= SP`toSlotRules[3, {}]
Out[*]:= {}
```

emptySymmetry

```
In[*]:= SP`emptySymmetry[x]
Out[*]:= xSymmetry[0, x, {}, GenSet[]]
```

```
In[*]:= SP`emptySymmetry[x, {a, b}]
Out[*]:= xSymmetry[2, x, {●1 → a, ●2 → b}, GenSet[]]
```

displaceSlots

CyclesSigned 표기의 순열에 있는 숫자 값들을 두 번째 인자 만큼 이동시킨다.

```
In[*]:= GenSet[{Cycles[{{1, 2}}], +1}, {Cycles[{{3, 4}}], +1}]
Out[*]:= SP`displaceSlots[%, 3]
GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{3, 4}}], 1}]
```

```
Out[*]:= GenSet[{Cycles[{{4, 5}}], 1}, {Cycles[{{6, 7}}], 1}]
```

```
In[*]:= SP`xSymmetryOf[RicciCD[la, lb]]
Out[*]:= SP`displaceSlots[%, 4]
xSymmetry[2, R●1●2, {●1 → la, ●2 → lb}, GenSet[{Cycles[{{1, 2}}], 1}]]
Out[*]:= xSymmetry[6, R●5●6, {●5 → la, ●6 → lb}, GenSet[{Cycles[{{5, 6}}], 1}]]
```

```
In[*]:= xSlot[2]
Out[*]:= SP`displaceSlots[%, 4]
●2
Out[*]:= ●6
```

commutingCycles

xSymmetryOf 함수를 xCommutingObjects에 적용했을 때 사용

```
In[*]:= SP`commutingCycles[1, 1]
Out[*]=
```

```
GenSet[]
```

```
In[*]:= SP`commutingCycles[4, 1]
Out[*]=
```

```
GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}, {Cycles[{{3, 4}}], 1}]
```

```
In[*]:= SP`commutingCycles[6, 2]
Out[*]=
```

```
GenSet[{Cycles[{{1, 3}, {2, 4}}], 1}, {Cycles[{{3, 5}, {4, 6}}], 1}]
```

joinSymmetries

두 개의 xSymmetry를 합한다. 이 때 두 번째 인자에 displaceSlots 명령이 적용된다.

```
In[*]:= s1 = xSymmetry[2, T[xSlot[1], xSlot[2]],
  {xSlot[1] → A, xSlot[2] → B}, GenSet[{Cycles[{{1, 2}}], +1}]]
s2 = xSymmetry[3, T[xSlot[1], xSlot[2], xSlot[3]],
  {xSlot[1] → C, xSlot[2] → D, xSlot[3] → F}, GenSet[{Cycles[{{1, 2}}], +1}]]
SP`joinSymmetries[s1, s2]
```

```
Out[*]= xSymmetry[2, T12, {1 → A, 2 → B}, GenSet[{Cycles[{{1, 2}}], 1}]]
```

```
Out[*]= xSymmetry[3, T123, {1 → C, 2 → D, 3 → F}, GenSet[{Cycles[{{1, 2}}], 1}]]
```

```
Out[*]= xSymmetry[5, xTTimes[T12, T345], {1 → A, 2 → B, 3 → C, 4 → D, 5 → F},
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{3, 4}}], 1}]]
```

```
In[*]:= s3 = SP`xSymmetryOf[Tscalar[T[la, lb]]2]
SP`joinSymmetries[s1, s3]
```

```
Out[*]= xSymmetry[0, (Tab)2, {}, GenSet[]]
```

```
Out[*]= xSymmetry[2, xTTimes[T12, (Tab)2],
  {1 → A, 2 → B}, GenSet[{Cycles[{{1, 2}}], 1}]]
```

IndexedOperand

IndexedOperand의 GenSet 대칭을 입력한 인덱스를 바탕으로 xSymmetry 표기로 바꾼다:

```
In[*]:= SP`xSymmetryOf[RicciCD[A, B]]
Out[*]:= xSymmetry[2, R12, {1 → A, 2 → B}, GenSet[{Cycles[{1, 2}], 1}]]
```

```
In[*]:= SP`xSymmetryOf[RiemannCD[G, F, D, K]]
Out[*]:= xSymmetry[4, R1234,
  {1 → G, 2 → F, 3 → D, 4 → K}, GenSet[{Cycles[{1, 2}], -1},
    {Cycles[{3, 4}], -1}, {Cycles[{1, 3}, {2, 4}], 1}]]
```

IndexedOperator

Non-CD type operators

CD type이 아닌 연산자 표현은 피 연산자에 xSymmetryOf를 적용한 후 xSymmetry의 두 번째 요소에 그 연산자를 작용한 표현으로 바꾼다:

```
In[*]:= SP`defineOperator[someD, "D", XD]
In[*]:= SP`xSymmetryOf[RiemannCD[G, F, D, K]]
Out[*]:= xSymmetry[4, R1234,
  {1 → G, 2 → F, 3 → D, 4 → K}, GenSet[{Cycles[{1, 2}], -1},
    {Cycles[{3, 4}], -1}, {Cycles[{1, 3}, {2, 4}], 1}]]
```

```
In[*]:= SP`xSymmetryOf[someD[RiemannCD[G, F, D, K]]]
Out[*]:= xSymmetry[4, DR1234,
  {1 → G, 2 → F, 3 → D, 4 → K}, GenSet[{Cycles[{1, 2}], -1},
    {Cycles[{3, 4}], -1}, {Cycles[{1, 3}, {2, 4}], 1}]]
```

processDerStackSym

processDerStack 함수에서 사용한다. CD type 연산자가 연속해서 작용하여 연산자 인덱스들이 서로 교환 가능할 때, 그 연산자 인덱스의 교환 대칭을 추가한다. 동일한 이름의 연속한 연산자는 첫 번째 tmpCovD에 포함되어 있다. 본질적으로 BD 연산자를 고려한 함수이다.

동일한 연산자가 한 개 뿐이다:

```
In[*]:= expr = .
```



```
In[*]:= SP`processDerStackSym[
  xSymmetry[n, expr, rules, GS, SP`tmpCovD[CovD[la]], SP`tmpCovD[z]]]
```

```
Out[*]= mGRG`STensor`Private`processDerStack[xSymmetry[1 + n, D1expr,
  Join[{1 → la}, rules], GS, mGRG`STensor`Private`tmpCovD[z]]]
```

```
In[*]:= SP`processDerStackSym[
  xSymmetry[n, expr, rules, GS, SP`tmpCovD[BD[la]], SP`tmpCovD[z]]]
```

```
Out[*]= mGRG`STensor`Private`processDerStack[xSymmetry[1 + n,  $\hat{D}_1$ expr,
  Join[{1 → la}, rules], GS, mGRG`STensor`Private`tmpCovD[z]]]
```

동일한 연산자가 두 개 이지만 연산자 인덱스가 교환되지 않는 경우:

```
In[*]:= SP`processDerStackSym[
  xSymmetry[n, expr, rules, GS, SP`tmpCovD[CovD[lb], CovD[la]], SP`tmpCovD[z]]]
```

```
Out[*]= mGRG`STensor`Private`processDerStack[xSymmetry[2 + n, D1D2expr,
  Join[{1 → la, 2 → lb}, rules], GS, mGRG`STensor`Private`tmpCovD[z]]]
```

BD 연산자가 두 개이고, 연산자 인덱스가 교환되는 경우:

```
In[*]:= SP`processDerStackSym[
  xSymmetry[n, expr, rules, GS, SP`tmpCovD[BD[lb], BD[la]], SP`tmpCovD[z]]]
```

```
Out[*]= mGRG`STensor`Private`processDerStack[
  xSymmetry[2 + n,  $\hat{D}_1\hat{D}_2$ expr, Join[{1 → la, 2 → lb}, rules],
  Join[GenSet[{Cycles[{1, 2}]}], 1], GS], mGRG`STensor`Private`tmpCovD[z]]]
```

BD 연산자가 두 개이지만 연산자 인덱스가 {up, dn}이기 때문에 교환되지 않는 경우:

```
In[*]:= SP`processDerStackSym[
  xSymmetry[n, expr, rules, GS, SP`tmpCovD[BD[ub], BD[la]], SP`tmpCovD[z]]]
```

```
Out[*]= mGRG`STensor`Private`processDerStack[xSymmetry[2 + n,  $\hat{D}_1\hat{D}_2$ expr,
  Join[{1 → la, 2 → ub}, rules], GS, mGRG`STensor`Private`tmpCovD[z]]]
```

BD 연산자가 두 개이지만 연산자 인덱스가 {dn, up}이기 때문에 교환되지 않는 경우:

```
In[*]:= SP`processDerStackSym[
  xSymmetry[n, expr, rules, GS, SP`tmpCovD[BD[lb], BD[ua]], SP`tmpCovD[z]]]
```

```
Out[*]= mGRG`STensor`Private`processDerStack[xSymmetry[2 + n,  $\hat{D}_1\hat{D}_2$ expr,
  Join[{1 → ua, 2 → lb}, rules], GS, mGRG`STensor`Private`tmpCovD[z]]]
```

BD 연산자가 두 개이고 연산자 인덱스가 {dn, up}이지만 CovDs 옵션을 조정하여 BD 연산자를 공변 연산자로

취급하였기 때문에 인덱스의 dn/up 상태와 관계없이 교환 가능한 경우:

```
In[*]:= SP`processDerStackSym[xSymmetry[n, expr, rules, GS,
  SP`tmpCovD[BD[lb], BD[ua]], SP`tmpCovD[z]], CovDs -> {BD}]

Out[*]:= mGRG`STensor`Private`processDerStack[xSymmetry[2 + n,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2}$  expr,
  Join[{ $\bullet 1 \rightarrow ua$ ,  $\bullet 2 \rightarrow lb$ }, rules], Join[GenSet[{Cycles[{1, 2}]}], 1]], GS],
  mGRG`STensor`Private`tmpCovD[z]], CovDs -> {BD}]
```

BD 연산자가 세 개이고 연산자 인덱스가 교환되는 경우:

```
In[*]:= SP`processDerStackSym[xSymmetry[n, expr, rules,
  GS, SP`tmpCovD[BD[lc], BD[lb], BD[la]], SP`tmpCovD[z]]]

Out[*]:= mGRG`STensor`Private`processDerStack[
  xSymmetry[3 + n,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} \hat{\partial}_{\bullet 3}$  expr, Join[{ $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow lb$ ,  $\bullet 3 \rightarrow lc$ }, rules],
  Join[GenSet[{Cycles[{1, 2}]}], 1], {Cycles[{2, 3}]}], 1]], GS],
  mGRG`STensor`Private`tmpCovD[z]]]
```

아래 표현에 대한 결과는 **잘못되었다**. BD[la, ub, lc, expr]에서 연산자 인덱스는 대칭이 없어야 함에도 불구하고 la와 lc 사이에 교환 대칭이 있는 것으로 출력되었다.

```
In[*]:= SP`processDerStackSym[xSymmetry[n, expr, rules,
  GS, SP`tmpCovD[BD[lc], BD[ub], BD[la]], SP`tmpCovD[z]]]

Out[*]:= mGRG`STensor`Private`processDerStack[
  xSymmetry[3 + n,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} \hat{\partial}_{\bullet 3}$  expr, Join[{ $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow ub$ ,  $\bullet 3 \rightarrow lc$ }, rules],
  Join[GenSet[{Cycles[{1, 3}]}], 1]], GS], mGRG`STensor`Private`tmpCovD[z]]]
```

그러나 processDerStackSym 함수는 processDerStack 함수가 호출하는 것이고, 위와 같은 입력을 갖는 processDerStackSym 함수의 호출은 processDerStack 함수의 정의

```
processDerStack[..]:=
  processDerStackSym[.., tmpCovD[cd_[_][..], cd_[a_], ____], ...] /; !covariantNameQ[..] && UpIndexQ[a]
```

때문에 불가능하다:

```
In[*]:= processDerStack0[tmpCovD0[y : covd[_] .., covd_[a_], z___]] :=
  processDerStackSym0[tmpCovD0[y], tmpCovD0[covd[a], z]] /; UpIndexQ[a];
processDerStack0[tmpCovD0[y : _[_] ..]] :=
  processDerStackSym0[tmpCovD0[y], tmpCovD0[]]

In[*]:= processDerStack0[tmpCovD0[cd[la], cd[lb], cd[lc]]]

Out[*]:= processDerStackSym0[tmpCovD0[cd[la], cd[lb], cd[lc]], tmpCovD0[]]
```

In[*]:= processDerStack0[tmpCovD0[cd[la], cd[lb], cd[uc]]]

Out[*]:= processDerStackSym0[tmpCovD0[cd[la], cd[lb]], tmpCovD0[cd[uc]]]

In[*]:= processDerStack0[tmpCovD0[cd[la], cd[ub], cd[lc]]]

Out[*]:= processDerStackSym0[tmpCovD0[cd[la]], tmpCovD0[cd[ub], cd[lc]]]

In[*]:= processDerStack0[tmpCovD0[cd[ua], cd[lb], cd[lc]]]

Out[*]:= processDerStackSym0[tmpCovD0[cd[ua], cd[lb], cd[lc]], tmpCovD0[]]

즉, 첫 번째 tmpCovD에 포함되는 연산자 표현에 {dn, up, dn} 형태는 불가능하다.

processDerStack

cdTypeSymmetryOf 함수에서 사용한다.

규칙 1: tmpCovD가 비어 있으면 xSymmetry를 그대로 돌려 준다.

In[*]:= SP`processDerStack[xSymmetry[n, expr, rules, gs, SP`tmpCovD[]], opts]

Out[*]:= xSymmetry[n, expr, rules, gs]

규칙 2: tmpCovD의 첫 번째 연산자가 나머지와 교환되지 않으면 첫 번째 연산자를 제거한다.

첫 번째 연산자가 BD가 아니고 미분 연산자이면 교환되지 않는다. 또한 BD이면서 공변 도함수로 설정되지 않았고, 연산자 인덱스가 upper 인덱스이면 교환되지 않는다:

In[*]:= SP`processDerStack[xSymmetry[n, expr, rules, gs, SP`tmpCovD[CovD[a]]]]

Out[*]:= xSymmetry[1 + n, D₁expr, Join[{ $\bullet 1 \rightarrow a$ }, rules], gs]

In[*]:= SP`processDerStack[xSymmetry[n, expr, rules, gs, SP`tmpCovD[BD[ua]]]]

Out[*]:= xSymmetry[1 + n, $\hat{D}_{1$ expr, Join[{ $\bullet 1 \rightarrow ua$ }, rules], gs]

규칙 3: tmpCovD의 첫 번째와 두 번째 연산자가 서로 다르면 첫 번째 연산자를 제거한다.

In[*]:= SP`processDerStack[xSymmetry[n, expr, rules, gs, SP`tmpCovD[CovD[a], BD[b]]]]

Out[*]:= xSymmetry[2 + n, $\hat{D}_{1}D_{2}$ expr, Join[{ $\bullet 1 \rightarrow b$ }, { $\bullet 2 \rightarrow a$ }, rules], gs]

규칙 4: TorsionFreeQ를 만족하는 CD type 연산자가 스칼라에 연속적으로 작용하면 그 연산자 인덱스들은 서로 교환된다:

```
In[*]:= SP`processDerStack[xSymmetry[n, f[], {}], gs, SP`tmpCovD[CD[lb], CD[la]]]]
Out[*]=
```

```
xSymmetry[2 + n,  $\nabla_{\bullet 1} \nabla_{\bullet 2} f$ , { $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow lb$ },
Join[GenSet[{Cycles[{1, 2}]}], 1]], gs]]
```

규칙 5: TorsionFreeQ를 만족하지 않는 CD type 연산자는 스칼라에 연속적으로 작용하여도 그 연산자 인덱스들은 서로 교환되지 않는다:

```
In[*]:= DefDerivativeOperator[CovD, "D", TorsionFreeQ  $\rightarrow$  False]
```

```
In[*]:= SP`processDerStack[xSymmetry[n, f[], {}], gs, SP`tmpCovD[CovD[lb], CovD[la]]]]
Out[*]=
```

```
xSymmetry[2 + n,  $D_{\bullet 1} D_{\bullet 2} f$ , { $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow lb$ }, gs]
```

기본인 TorsionFree-CovD로 복귀한다:

```
In[*]:= DefDerivativeOperator[CovD, "D"] (* default *)
```

```
SP`processDerStack[
  xSymmetry[n, f[], {}], gs, SP`tmpCovD[CovD[lb], CovD[la]]]]
Out[*]=
```

```
xSymmetry[2 + n,  $D_{\bullet 1} D_{\bullet 2} f$ , { $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow lb$ },
Join[GenSet[{Cycles[{1, 2}]}], 1]], gs]]
```

규칙 6: tmpCovD에 동일한 연산자가 한 개 이상 있고 다른 연산자가 포함되었을 때 processDerStackSym 함수를 호출하여 해결한다:

```
In[*]:= SP`processDerStack[xSymmetry[n, expr, {}], gs, SP`tmpCovD[CovD[la], CD[lb]]]]
Out[*]=
```

```
xSymmetry[2 + n,  $\nabla_{\bullet 1} D_{\bullet 2} expr$ , { $\bullet 1 \rightarrow lb$ ,  $\bullet 2 \rightarrow la$ }, gs]
```

규칙 7: tmpCovD에 동일한 연산자가 연속해서 있을 때 processDerStackSym 함수를 호출한다. 특히 processDerStackSym 함수는 연속한 BD 연산자의 인덱스 대칭에 대한 교환 대칭을 다룬다. 이 때 processDerStack 함수에서 조건 UpIndexQ[a]를 사용했기 때문에 다음과 같은 연속한 BD 연산자의 대칭도 제대로 다룰 수 있다 (processDerStackSym 함수 참조):

```
In[*]:= SP`processDerStack[
  xSymmetry[n, expr, rules, gs, SP`tmpCovD[BD[lc], BD[ub], BD[la]]]]
Out[*]=
```

```
xSymmetry[3 + n,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} \hat{\partial}_{\bullet 3} expr$ , Join[{ $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow ub$ }, { $\bullet 3 \rightarrow lc$ }, rules], gs]
```

```
In[*]:= SP`processDerStack[
  xSymmetry[n, expr, rules, GenSet[], SP`tmpCovD[BD[lc], BD[lb], BD[ua]]]]
Out[*]=
```

```
xSymmetry[3 + n,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} \hat{\partial}_{\bullet 3} expr$ ,
Join[{ $\bullet 1 \rightarrow ua$ }, { $\bullet 2 \rightarrow lb$ ,  $\bullet 3 \rightarrow lc$ }, rules], GenSet[{Cycles[{2, 3}]}], 1]]]
```

```

In[*]:= SP`processDerStack[
  xSymmetry[n, expr, rules, GenSet[], SP`tmpCovD[BD[uc], BD[lb], BD[la]]]]
Out[*]=
  xSymmetry[3 + n,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} \hat{\partial}_{\bullet 3} \text{expr}$ ,
    Join[{ $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow lb$ ,  $\bullet 3 \rightarrow uc$ }, rules], GenSet[{Cycles[{1, 2}], 1}]]

```

Symmetries of CD[la,lb,scale]:

RiemannCD는 스칼라가 아니고 곡률 텐서의 대칭을 갖지만 RiemannCD[G,F,D,K] 표현에서 사용한 인덱스가 Latin 계열이 아니므로 연산자 입장에서는 스칼라로 간주한다:

```

In[*]:= FindIndices /@ {RiemannCD[G, F, D, K], RiemannCD[ug, F, D, K]}
Out[*]=
  {{}, {ug}}

```

```

In[*]:= NoIndexQ /@ {RiemannCD[G, F, D, K], RiemannCD[ug, F, D, K]}
Out[*]=
  {True, False}

```

한 개의 CD type 연산자로는 추가되는 대칭이 없다:

```

In[*]:= SP`xSymmetryOf[BD[la, RiemannCD[G, F, D, K]]]
Out[*]=
  xSymmetry[5,  $\hat{\partial}_{\bullet 1} R_{\bullet 2 \bullet 3 \bullet 4 \bullet 5}$ ,
    { $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow G$ ,  $\bullet 3 \rightarrow F$ ,  $\bullet 4 \rightarrow D$ ,  $\bullet 5 \rightarrow K$ }, GenSet[{Cycles[{2, 3}], -1},
      {Cycles[{4, 5}], -1}, {Cycles[{2, 4}, {3, 5}], 1}]]

```

```

In[*]:= SP`xSymmetryOf[CD[la, RiemannCD[G, F, D, K]]]
Out[*]=
  xSymmetry[5,  $\nabla_{\bullet 1} R_{\bullet 2 \bullet 3 \bullet 4 \bullet 5}$ ,
    { $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow G$ ,  $\bullet 3 \rightarrow F$ ,  $\bullet 4 \rightarrow D$ ,  $\bullet 5 \rightarrow K$ }, GenSet[{Cycles[{2, 3}], -1},
      {Cycles[{4, 5}], -1}, {Cycles[{2, 4}, {3, 5}], 1}]]

```

BD와 CD는 서로 교환되지 않는다:

```

In[*]:= SP`xSymmetryOf[BD[la, CD[lb, RiemannCD[G, F, D, K]]]]
Out[*]=
  xSymmetry[6,  $\hat{\partial}_{\bullet 1} \nabla_{\bullet 2} R_{\bullet 3 \bullet 4 \bullet 5 \bullet 6}$ ,
    { $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow lb$ ,  $\bullet 3 \rightarrow G$ ,  $\bullet 4 \rightarrow F$ ,  $\bullet 5 \rightarrow D$ ,  $\bullet 6 \rightarrow K$ }, GenSet[{Cycles[{3, 4}], -1},
      {Cycles[{5, 6}], -1}, {Cycles[{3, 5}, {4, 6}], 1}]]

```

연산자 CD 입장에서 RiemannCD[G,F,D,K]는 스칼라이고, CD는 TorsionFreeQ를 만족하므로 연속한 CD 연산자의 인덱스 la와 lb는 서로 교환된다:

```
In[*]:= SP`xSymmetryOf[CD[la, lb, RiemannCD[G, F, D, K]]]
Out[*]= xSymmetry[6,  $\nabla_{\bullet 1} \nabla_{\bullet 2} R_{\bullet 3 \bullet 4 \bullet 5 \bullet 6}$ , { $\bullet 1 \rightarrow la, \bullet 2 \rightarrow lb, \bullet 3 \rightarrow G, \bullet 4 \rightarrow F, \bullet 5 \rightarrow D, \bullet 6 \rightarrow K$ },
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{3, 4}}], -1},
    {Cycles[{{5, 6}}], -1}, {Cycles[{{3, 5}, {4, 6}}], 1}]]
```

같은 이유로 인덱스 lb와 lc는 서로 교환된다:

```
In[*]:= SP`xSymmetryOf[CD[la, lb, lc, RiemannCD[G, F, D, K]]]
Out[*]= xSymmetry[7,  $\nabla_{\bullet 1} \nabla_{\bullet 2} \nabla_{\bullet 3} R_{\bullet 4 \bullet 5 \bullet 6 \bullet 7}$ ,
  { $\bullet 1 \rightarrow la, \bullet 2 \rightarrow lb, \bullet 3 \rightarrow lc, \bullet 4 \rightarrow G, \bullet 5 \rightarrow F, \bullet 6 \rightarrow D, \bullet 7 \rightarrow K$ },
  GenSet[{Cycles[{{2, 3}}], 1}, {Cycles[{{4, 5}}], -1},
    {Cycles[{{6, 7}}], -1}, {Cycles[{{4, 6}, {5, 7}}], 1}]]
```

CD는 공변 연산자이므로 비록 인덱스 ub와 lc가 {up, dn} 일지라도 서로 교환된다:

```
In[*]:= SP`xSymmetryOf[CD[la, ub, lc, RiemannCD[G, F, D, K]]]
Out[*]= xSymmetry[7,  $\nabla_{\bullet 1} \nabla_{\bullet 2} \nabla_{\bullet 3} R_{\bullet 4 \bullet 5 \bullet 6 \bullet 7}$ ,
  { $\bullet 1 \rightarrow la, \bullet 2 \rightarrow ub, \bullet 3 \rightarrow lc, \bullet 4 \rightarrow G, \bullet 5 \rightarrow F, \bullet 6 \rightarrow D, \bullet 7 \rightarrow K$ },
  GenSet[{Cycles[{{2, 3}}], 1}, {Cycles[{{4, 5}}], -1},
    {Cycles[{{6, 7}}], -1}, {Cycles[{{4, 6}, {5, 7}}], 1}]]
```

연산자 입장에서 RiemannCD[ug,F,D,K]는 스칼라가 아니므로 TorsionFreeQ를 만족하는 공변도함수일지라도 추가되는 인덱스 대칭이 없다:

```
In[*]:= SP`xSymmetryOf[CD[la, lb, lc, RiemannCD[ug, F, D, K]]]
Out[*]= xSymmetry[7,  $\nabla_{\bullet 1} \nabla_{\bullet 2} \nabla_{\bullet 3} R_{\bullet 4 \bullet 5 \bullet 6 \bullet 7}$ ,
  { $\bullet 1 \rightarrow la, \bullet 2 \rightarrow lb, \bullet 3 \rightarrow lc, \bullet 4 \rightarrow ug, \bullet 5 \rightarrow F, \bullet 6 \rightarrow D, \bullet 7 \rightarrow K$ },
  GenSet[{Cycles[{{4, 5}}], -1},
    {Cycles[{{6, 7}}], -1}, {Cycles[{{4, 6}, {5, 7}}], 1}]]
```

공변 연산자가 TorsionFreeQ를 만족하지 않는다면 비록 스칼라에 연속적으로 작용해도 추가되는 인덱스 대칭은 없다:

```
In[*]:= TorsionFreeQ[CD] = False;
In[*]:= SP`xSymmetryOf[CD[la, ub, RiemannCD[G, F, D, K]]]
Out[*]= xSymmetry[6,  $\nabla_{\bullet 1} \nabla_{\bullet 2} R_{\bullet 3 \bullet 4 \bullet 5 \bullet 6}$ ,
  { $\bullet 1 \rightarrow la, \bullet 2 \rightarrow ub, \bullet 3 \rightarrow G, \bullet 4 \rightarrow F, \bullet 5 \rightarrow D, \bullet 6 \rightarrow K$ }, GenSet[{Cycles[{{3, 4}}], -1},
    {Cycles[{{5, 6}}], -1}, {Cycles[{{3, 5}, {4, 6}}], 1}]]
```

기본 TorsionFree-CD로 복귀한다:

```

In[*]:= TorsionFreeQ[CD] = True;
SP`xSymmetryOf[CD[la, ub, RiemannCD[G, F, D, K]]]

Out[*]=
xSymmetry[6,  $\nabla_{\bullet 1} \nabla_{\bullet 2} R_{\bullet 3 \bullet 4 \bullet 5 \bullet 6}$ , { $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow ub$ ,  $\bullet 3 \rightarrow G$ ,  $\bullet 4 \rightarrow F$ ,  $\bullet 5 \rightarrow D$ ,  $\bullet 6 \rightarrow K$ },
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{3, 4}}], -1},
    {Cycles[{{5, 6}}], -1}, {Cycles[{{3, 5}, {4, 6}}], 1}]

```

Symmetries of BD[la,lb,someT]:

비좌표 기준에서는 BD 연산자가 연속하여 작용하여도 더 이상 추가되는 인덱스 대칭이 없다:

```

In[*]:= Off[CoordinateBasisFlag]

In[*]:= SP`xSymmetryOf[BD[la, lb, lc, RiemannCD[G, F, D, K]]]

Out[*]=
xSymmetry[7,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} \hat{\partial}_{\bullet 3} R_{\bullet 4 \bullet 5 \bullet 6 \bullet 7}$ ,
  { $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow lb$ ,  $\bullet 3 \rightarrow lc$ ,  $\bullet 4 \rightarrow G$ ,  $\bullet 5 \rightarrow F$ ,  $\bullet 6 \rightarrow D$ ,  $\bullet 7 \rightarrow K$ },
  GenSet[{Cycles[{{4, 5}}], -1},
    {Cycles[{{6, 7}}], -1}, {Cycles[{{4, 6}, {5, 7}}], 1}]

```

좌표 기준에서 연속된 BD 연산자의 인덱스는 교환 가능하다:

```

In[*]:= On[CoordinateBasisFlag]

In[*]:= SP`xSymmetryOf[BD[la, lb, RiemannCD[G, F, D, K]]]

Out[*]=
xSymmetry[6,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} R_{\bullet 3 \bullet 4 \bullet 5 \bullet 6}$ , { $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow lb$ ,  $\bullet 3 \rightarrow G$ ,  $\bullet 4 \rightarrow F$ ,  $\bullet 5 \rightarrow D$ ,  $\bullet 6 \rightarrow K$ },
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{3, 4}}], -1},
    {Cycles[{{5, 6}}], -1}, {Cycles[{{3, 5}, {4, 6}}], 1}]

```

BD 연산자는 공변 연산자가 아니므로 인덱스가 {up, dn}이면 교환되지 않는다:

```

In[*]:= SP`xSymmetryOf[BD[ua, lb, RiemannCD[G, F, D, K]]]

Out[*]=
xSymmetry[6,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} R_{\bullet 3 \bullet 4 \bullet 5 \bullet 6}$ ,
  { $\bullet 1 \rightarrow ua$ ,  $\bullet 2 \rightarrow lb$ ,  $\bullet 3 \rightarrow G$ ,  $\bullet 4 \rightarrow F$ ,  $\bullet 5 \rightarrow D$ ,  $\bullet 6 \rightarrow K$ }, GenSet[{Cycles[{{3, 4}}], -1},
    {Cycles[{{5, 6}}], -1}, {Cycles[{{3, 5}, {4, 6}}], 1}]

```

인덱스가 {dn, up}인 경우도 마찬가지이다:

```

In[*]:= SP`xSymmetryOf[BD[la, ub, RiemannCD[G, F, D, K]]]

Out[*]=
xSymmetry[6,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} R_{\bullet 3 \bullet 4 \bullet 5 \bullet 6}$ ,
  { $\bullet 1 \rightarrow la$ ,  $\bullet 2 \rightarrow ub$ ,  $\bullet 3 \rightarrow G$ ,  $\bullet 4 \rightarrow F$ ,  $\bullet 5 \rightarrow D$ ,  $\bullet 6 \rightarrow K$ }, GenSet[{Cycles[{{3, 4}}], -1},
    {Cycles[{{5, 6}}], -1}, {Cycles[{{3, 5}, {4, 6}}], 1}]

```

CovDs 옵션을 조정하여 BD 연산자를 공변 연산자로 만들면 비록 인덱스가 {up, dn}, {dn, up}, 또는 {up,

up}일지라도 서로 교환 가능하다:

```
In[*]:= SP`xSymmetryOf[BD[la, ub, RiemannCD[G, F, D, K]], CovDs -> {CD, BD}]
Out[*]:= xSymmetry[6,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} R_{\bullet 3 \bullet 4 \bullet 5 \bullet 6}$ , { $\bullet 1 \rightarrow la, \bullet 2 \rightarrow ub, \bullet 3 \rightarrow G, \bullet 4 \rightarrow F, \bullet 5 \rightarrow D, \bullet 6 \rightarrow K$ },
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{3, 4}}], -1},
    {Cycles[{{5, 6}}], -1}, {Cycles[{{3, 5}, {4, 6}}], 1}]]
```

```
In[*]:= SP`xSymmetryOf[BD[ua, lb, RiemannCD[G, F, D, K]], CovDs -> {CD, BD}]
Out[*]:= xSymmetry[6,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} R_{\bullet 3 \bullet 4 \bullet 5 \bullet 6}$ , { $\bullet 1 \rightarrow ua, \bullet 2 \rightarrow lb, \bullet 3 \rightarrow G, \bullet 4 \rightarrow F, \bullet 5 \rightarrow D, \bullet 6 \rightarrow K$ },
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{3, 4}}], -1},
    {Cycles[{{5, 6}}], -1}, {Cycles[{{3, 5}, {4, 6}}], 1}]]
```

```
In[*]:= SP`xSymmetryOf[BD[ua, ub, RiemannCD[G, F, D, K]], CovDs -> {CD, BD}]
Out[*]:= xSymmetry[6,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} R_{\bullet 3 \bullet 4 \bullet 5 \bullet 6}$ , { $\bullet 1 \rightarrow ua, \bullet 2 \rightarrow ub, \bullet 3 \rightarrow G, \bullet 4 \rightarrow F, \bullet 5 \rightarrow D, \bullet 6 \rightarrow K$ },
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{3, 4}}], -1},
    {Cycles[{{5, 6}}], -1}, {Cycles[{{3, 5}, {4, 6}}], 1}]]
```

연속된 BD 연산자의 인덱스 {la, lb, lc} 모두가 교환 가능하다:

```
In[*]:= SP`xSymmetryOf[BD[la, lb, lc, RiemannCD[G, F, D, K]]]
Out[*]:= xSymmetry[7,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} \hat{\partial}_{\bullet 3} R_{\bullet 4 \bullet 5 \bullet 6 \bullet 7}$ ,
  { $\bullet 1 \rightarrow la, \bullet 2 \rightarrow lb, \bullet 3 \rightarrow lc, \bullet 4 \rightarrow G, \bullet 5 \rightarrow F, \bullet 6 \rightarrow D, \bullet 7 \rightarrow K$ },
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}, {Cycles[{{4, 5}}], -1},
    {Cycles[{{6, 7}}], -1}, {Cycles[{{4, 6}, {5, 7}}], 1}]]
```

비록 BD 연산자가 연속되었을지라도 인덱스 {la, ub, lc}는 모두 교환 가능하지 않다:

```
In[*]:= SP`xSymmetryOf[BD[la, ub, lc, RiemannCD[G, F, D, K]]]
Out[*]:= xSymmetry[7,  $\hat{\partial}_{\bullet 1} \hat{\partial}_{\bullet 2} \hat{\partial}_{\bullet 3} R_{\bullet 4 \bullet 5 \bullet 6 \bullet 7}$ ,
  { $\bullet 1 \rightarrow la, \bullet 2 \rightarrow ub, \bullet 3 \rightarrow lc, \bullet 4 \rightarrow G, \bullet 5 \rightarrow F, \bullet 6 \rightarrow D, \bullet 7 \rightarrow K$ },
  GenSet[{Cycles[{{4, 5}}], -1},
    {Cycles[{{6, 7}}], -1}, {Cycles[{{4, 6}, {5, 7}}], 1}]]
```

BD 연산자가 공변 도함수이면 인덱스 {la, ub, lc} 모두가 교환 가능하다:


```
In[*]:= SP`xSymmetryOf[BD[la, ub, lc, RiemannCD[G, F, D, K]], CovDs -> {CD, BD}]
Out[*]=
```

```
xSymmetry[7,  $\hat{\partial}_{1\bullet}\hat{\partial}_{2\bullet}\hat{\partial}_{3\bullet}R_{\bullet4\bullet5\bullet6\bullet7}$ ,
  { $\bullet1 \rightarrow la, \bullet2 \rightarrow ub, \bullet3 \rightarrow lc, \bullet4 \rightarrow G, \bullet5 \rightarrow F, \bullet6 \rightarrow D, \bullet7 \rightarrow K$ },
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}, {Cycles[{{4, 5}}], -1},
    {Cycles[{{6, 7}}], -1}, {Cycles[{{4, 6}, {5, 7}}], 1}]]
```

인덱스 {la, lb}만 교환 가능하다:

```
In[*]:= SP`xSymmetryOf[BD[la, lb, uc, RiemannCD[G, F, D, K]]]
Out[*]=
```

```
xSymmetry[7,  $\hat{\partial}_{1\bullet}\hat{\partial}_{2\bullet}\hat{\partial}_{3\bullet}R_{\bullet4\bullet5\bullet6\bullet7}$ ,
  { $\bullet1 \rightarrow la, \bullet2 \rightarrow lb, \bullet3 \rightarrow uc, \bullet4 \rightarrow G, \bullet5 \rightarrow F, \bullet6 \rightarrow D, \bullet7 \rightarrow K$ },
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{4, 5}}], -1},
    {Cycles[{{6, 7}}], -1}, {Cycles[{{4, 6}, {5, 7}}], 1}]]
```

인덱스 {lc, ld}만 교환 가능하다:

```
In[*]:= SP`xSymmetryOf[BD[la, ub, lc, ld, RiemannCD[G, F, D, K]]]
Out[*]=
```

```
xSymmetry[8,  $\hat{\partial}_{1\bullet}\hat{\partial}_{2\bullet}\hat{\partial}_{3\bullet}\hat{\partial}_{4\bullet}R_{\bullet5\bullet6\bullet7\bullet8}$ ,
  { $\bullet1 \rightarrow la, \bullet2 \rightarrow ub, \bullet3 \rightarrow lc, \bullet4 \rightarrow ld, \bullet5 \rightarrow G, \bullet6 \rightarrow F, \bullet7 \rightarrow D, \bullet8 \rightarrow K$ },
  GenSet[{Cycles[{{3, 4}}], 1}, {Cycles[{{5, 6}}], -1},
    {Cycles[{{7, 8}}], -1}, {Cycles[{{5, 7}, {6, 8}}], 1}]]
```

BD 연산자를 공변 도함수로 지정하면 인덱스 {la, lb, uc} 모두가 교환 가능하다:

```
In[*]:= SP`xSymmetryOf[BD[la, lb, uc, RiemannCD[G, F, D, K]], CovDs -> {CD, BD}]
Out[*]=
```

```
xSymmetry[7,  $\hat{\partial}_{1\bullet}\hat{\partial}_{2\bullet}\hat{\partial}_{3\bullet}R_{\bullet4\bullet5\bullet6\bullet7}$ ,
  { $\bullet1 \rightarrow la, \bullet2 \rightarrow lb, \bullet3 \rightarrow uc, \bullet4 \rightarrow G, \bullet5 \rightarrow F, \bullet6 \rightarrow D, \bullet7 \rightarrow K$ },
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}, {Cycles[{{4, 5}}], -1},
    {Cycles[{{6, 7}}], -1}, {Cycles[{{4, 6}, {5, 7}}], 1}]]
```

BD 연산자를 공변 도함수로 지정하면 인덱스 {la, lb, uc, ld} 모두가 교환 가능하다:

```
In[*]:= SP`xSymmetryOf[BD[la, lb, uc, ld, RiemannCD[G, F, D, K]], CovDs -> {CD, BD}]
Out[*]=
```

```
xSymmetry[8,  $\hat{\partial}_{1\bullet}\hat{\partial}_{2\bullet}\hat{\partial}_{3\bullet}\hat{\partial}_{4\bullet}R_{\bullet5\bullet6\bullet7\bullet8}$ ,
  { $\bullet1 \rightarrow la, \bullet2 \rightarrow lb, \bullet3 \rightarrow uc, \bullet4 \rightarrow ld, \bullet5 \rightarrow G, \bullet6 \rightarrow F, \bullet7 \rightarrow D, \bullet8 \rightarrow K$ },
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1},
    {Cycles[{{3, 4}}], 1}, {Cycles[{{5, 6}}], -1},
    {Cycles[{{7, 8}}], -1}, {Cycles[{{5, 7}, {6, 8}}], 1}]]
```

BD 연산자는 모든 계열에서 사용된다. Latin 계열과 Greek 계열에서 BD 연산자를 사용해 보자.

Greek 계열은 기본적으로 좌표 기준이 아니다:

```
In[*]:= CoordinateBasisQ[Greek]
```

```
Out[*]:= False
```

Latin 계열은 좌표 기준이므로 인덱스 {la, lb}가 서로 교환된다. 그러나 서로 다른 계열의 인덱스는 서로 교환되지 않음을 주목하자:

```
In[*]:= SP`xSymmetryOf[BD[la, lb, lμ, ld, RiemannCD[G, F, D, K]]]
```

```
Out[*]:= xSymmetry[8, ∂1∂2∂3∂4R5678,
  {●1 → la, ●2 → lb, ●3 → lμ, ●4 → ld, ●5 → G, ●6 → F, ●7 → D, ●8 → K},
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{5, 6}}], -1},
    {Cycles[{{7, 8}}], -1}, {Cycles[{{5, 7}, {6, 8}}], 1}]]
```

Greek 계열은 비좌표 기준이므로 인덱스 {lμ, lv}는 서로 교환되지 않는다:

```
In[*]:= SP`xSymmetryOf[BD[la, lμ, lv, ld, RiemannCD[G, F, D, K]]]
```

```
Out[*]:= xSymmetry[8, ∂1∂2∂3∂4R5678,
  {●1 → la, ●2 → lμ, ●3 → lv, ●4 → ld, ●5 → G, ●6 → F, ●7 → D, ●8 → K},
  GenSet[{Cycles[{{5, 6}}], -1},
    {Cycles[{{7, 8}}], -1}, {Cycles[{{5, 7}, {6, 8}}], 1}]]
```

인덱스 {ld, le}가 서로 교환된다:

```
In[*]:= SP`xSymmetryOf[BD[la, lμ, lv, ld, le, RiemannCD[G, F, D, K]]]
```

```
Out[*]:= xSymmetry[9, ∂1∂2∂3∂4∂5R6789,
  {●1 → la, ●2 → lμ, ●3 → lv, ●4 → ld, ●5 → le, ●6 → G, ●7 → F, ●8 → D, ●9 → K},
  GenSet[{Cycles[{{4, 5}}], 1}, {Cycles[{{6, 7}}], -1},
    {Cycles[{{8, 9}}], -1}, {Cycles[{{6, 8}, {7, 9}}], 1}]]
```

서로 다른 계열의 인덱스는 교환되지 않는다:

```
In[*]:= SP`xSymmetryOf[BD[-1, lμ, lc, RiemannCD[G, F, D, K]]]
```

```
Out[*]:= xSymmetry[7, ∂1∂2∂3R4567,
  {●1 → -1, ●2 → lμ, ●3 → lc, ●4 → G, ●5 → F, ●6 → D, ●7 → K},
  GenSet[{Cycles[{{4, 5}}], -1},
    {Cycles[{{6, 7}}], -1}, {Cycles[{{4, 6}, {5, 7}}], 1}]]
```

```
In[*]:= SP`xSymmetryOf[BD[-1, lμ, lc, lv, RiemannCD[G, F, D, K]]]
Out[*]=
```

```
xSymmetry[8, ∂1∂2∂3∂4R5678,
{●1 → -1, ●2 → lμ, ●3 → lc, ●4 → lv, ●5 → G, ●6 → F, ●7 → D, ●8 → K},
GenSet[{Cycles[{{5, 6}}], -1},
{Cycles[{{7, 8}}], -1}, {Cycles[{{5, 7}, {6, 8}}], 1}]]
```

주목: 성분 인덱스는 기본적으로 DefaultKind 계열로 간주하므로 인덱스 {-l, lb}가 교환 가능하다:

```
In[*]:= SP`xSymmetryOf[BD[-1, lb, lμ, lc, RiemannCD[G, F, D, K]]]
Out[*]=
```

```
xSymmetry[8, ∂1∂2∂3∂4R5678,
{●1 → -1, ●2 → lb, ●3 → lμ, ●4 → lc, ●5 → G, ●6 → F, ●7 → D, ●8 → K},
GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{5, 6}}], -1},
{Cycles[{{7, 8}}], -1}, {Cycles[{{5, 7}, {6, 8}}], 1}]]
```

성분 인덱스 {-1, -2}는 교환된다:

```
In[*]:= SP`xSymmetryOf[BD[-1, -2, lc, RiemannCD[G, F, D, K]]]
Out[*]=
```

```
xSymmetry[7, ∂1∂2∂3R4567,
{●1 → -1, ●2 → -2, ●3 → lc, ●4 → G, ●5 → F, ●6 → D, ●7 → K},
GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}, {Cycles[{{4, 5}}], -1},
{Cycles[{{6, 7}}], -1}, {Cycles[{{4, 6}, {5, 7}}], 1}]]
```

인덱스 {-1, ub}는 교환되지 않는다:

```
In[*]:= SP`xSymmetryOf[BD[-1, ub, lμ, lc, RiemannCD[G, F, D, K]]]
Out[*]=
```

```
xSymmetry[8, ∂1∂2∂3∂4R5678,
{●1 → -1, ●2 → ub, ●3 → lμ, ●4 → lc, ●5 → G, ●6 → F, ●7 → D, ●8 → K},
GenSet[{Cycles[{{5, 6}}], -1},
{Cycles[{{7, 8}}], -1}, {Cycles[{{5, 7}, {6, 8}}], 1}]]
```

Others

ScalarFunction, errObject, \$flattenOTHER, xNoIndex에 대한 xSymmetryOf는 단순히 emptySymmetry를 사용한다:

```
In[*]:= SP`xSymmetryOf[Tscalar[T[la, lb]]]
Out[*]=
```

```
xSymmetry[0, (Tab), {}, GenSet[]]
```

```
In[*]:= SP`xSymmetryOf[Log[T[la, lb]]]
```

```
Out[*]= xSymmetry[0, Log[Tab], {}, GenSet[]]
```

```
In[*]:= SP`xSymmetryOf[Other]
```

```
Out[*]= xSymmetry[0, Other, {}, GenSet[]]
```

Product of objects

인자가 텐서 곱이면 xSort 함수를 먼저 적용하여 각각의 object들을 xTensorTimes가 돌려싼 표현을 얻는다. 그 후 각각의 object들에 독립적으로 xSymmetryOf를 적용하고, 결과적으로 얻어진 xSymmetry들을 joinSymmetries로 합친다.

```
In[*]:= SP`xSort[T[la, lb] × A[lc, ld]]
```

```
Out[*]= xTensorTimes[xObject[Acd, {A},
  {{lc, ld}, {lc, ld}, {}, {{-1, Metricg, 1}, {-1, Metricg, 1}}}], xObject[
  Tab, {T}, {{la, lb}, {la, lb}, {}, {{-1, Metricg, 1}, {-1, Metricg, 1}}]]
```

```
In[*]:= SP`xSymmetryOf /@ %
```

```
Out[*]= xTensorTimes[
  xSymmetry[2, A12, {1 → lc, 2 → ld}, GenSet[{Cycles[{1, 2}], -1}],
  xSymmetry[2, T12, {1 → la, 2 → lb}, GenSet[]]]
```

```
In[*]:= SP`joinSymmetries @@ %
```

```
Out[*]= xSymmetry[4, xTTimes[A12, T34],
  {1 → lc, 2 → ld, 3 → la, 4 → lb}, GenSet[{Cycles[{1, 2}], -1}]]
```

```
In[*]:= % == SP`xSymmetryOf[T[la, lb] × A[lc, ld]]
```

```
Out[*]= True
```

```
In[*]:= SP`xSymmetryOf[A[uc, ub] × RiemannCD[lj, lm, ll, lk]]
```

```
Out[*]= xSymmetry[6, xTTimes[A12, R3456],
  {1 → uc, 2 → ub, 3 → lj, 4 → lm, 5 → ll, 6 → lk},
  GenSet[{Cycles[{1, 2}], -1}, {Cycles[{3, 4}], -1},
  {Cycles[{5, 6}], -1}, {Cycles[{3, 5}, {4, 6}], 1}]]
```

xCommutingObjects

텐서 곱에 xSort 함수를 적용한 결과 xCommutingObjects가 얻어진 경우, xCommutingObjects를 구성하는 각각의 object들의 교환 대칭을 commutingCycles 함수를 사용하여 얻는다. 그리고 이 교환 대칭을 각 object들 자체가 갖고 있는 대칭과 합친다.

```
In[*]:= SP`xSort[v[la] × v[lb]]
Out[*]= xTensorTimes[
  xCommutingObjects[xObject[va, {v}, {{la}, {la}, {}, {{-1, Metricg, 1}}}],
  xObject[vb, {v}, {{lb}, {lb}, {}, {{-1, Metricg, 1}}}] ] ]
```

```
In[*]:= objL = List @@ %[[1]]
Out[*]= {xObject[va, {v}, {{la}, {la}, {}, {{-1, Metricg, 1}}}],
  xObject[vb, {v}, {{lb}, {lb}, {}, {{-1, Metricg, 1}}}] }
```

```
In[*]:= sym = SP`joinSymmetries @@ SP`xSymmetryOf /@ objL
Out[*]= xSymmetry[2, xTTimes[v1, v2], {●1 → la, ●2 → lb}, GenSet[]]
```

```
In[*]:= {nObj = Length[objL], nsTotal = Length@sym[[3]]}
Out[*]= {2, 2}
```

```
In[*]:= SP`commutingCycles[nsTotal, nsTotal / nObj]
Out[*]= GenSet[{Cycles[{{1, 2}}], 1}]
```

```
In[*]:= SP`xSymmetryOf[v[la] × v[lb]]
Out[*]= xSymmetry[2, xTTimes[v1, v2],
  {●1 → la, ●2 → lb}, GenSet[{Cycles[{{1, 2}}], 1}]]
```

아래 표현 $v_a v^1 v^b$ 에서는 v_a 와 v^b 의 교환 대칭만 추가된다. 성분 인덱스를 갖는 v^1 은 고려되지 않는 BUG가 발생한 것이다. 이 문제가 발생하는 이유는, v[1]은 성분 인덱스를 갖기 때문에 metricState는 {dnup, Null, _}이고, 따라서 mStateOrderItems가 (v[ua] 경우의 metric 대신에) dnup이 되기 때문이다. 결과적으로 objectOrderItems에서 v[ua]와 v[1]은 달리 취급된다.

In[*]:= SP`xSort[v[la] × v[1] × v[ub]]

Out[*]=

```
xTensorTimes[xObject[v1, {v}, {{1}, {1}, {}, {{1, Null, 0}}}],
  xCommutingObjects[xObject[va, {v}, {{la}, {la}, {}, {{-1, Metricg, 1}}}],
  xObject[vb, {v}, {{ub}, {ub}, {}, {{1, Metricg, 1}}}] ]]
```

In[*]:= SP`xSymmetryOf[v[la] × v[1] × v[ub]]

Out[*]=

```
xSymmetry[3, xTTimes[v1, v2, v3],
  {1 → 1, 2 → la, 3 → ub}, GenSet[{Cycles[{{2, 3}}], 1}]]
```

정규 인덱스와 성분 인덱스를 (당연하게) 구분하지 않고 교환 대칭을 구성한다:

In[*]:= SP`xSort[T[la, 1] × T[ub, 2]]

Out[*]=

```
xTensorTimes[xCommutingObjects[
  xObject[Ta1, {T}, {{la, 1}, {la, 1}, {}, {{-1, Metricg, 1}, {1, Null, 0}}}],
  xObject[Tb2, {T}, {{ub, 2}, {ub, 2}, {}, {{1, Metricg, 1}, {1, Null, 0}}}] ]]
```

In[*]:= SP`xSymmetryOf[T[la, 1] × T[ub, 2]]

Out[*]=

```
xSymmetry[4, xTTimes[T12, T34],
  {1 → la, 2 → 1, 3 → ub, 4 → 2}, GenSet[{Cycles[{{1, 3}, {2, 4}}], 1}]]
```

More examples:

In[*]:= SP`xSort[v[ua] × v[ub] × v[uc]]

Out[*]=

```
xTensorTimes[
  xCommutingObjects[xObject[va, {v}, {{ua}, {ua}, {}, {{1, Metricg, 1}}}],
  xObject[vb, {v}, {{ub}, {ub}, {}, {{1, Metricg, 1}}}],
  xObject[vc, {v}, {{uc}, {uc}, {}, {{1, Metricg, 1}}}] ]]
```

In[*]:= SP`xSymmetryOf[v[ua] × v[ub] × v[uc]]

Out[*]=

```
xSymmetry[3, xTTimes[v1, v2, v3], {1 → ua, 2 → ub, 3 → uc},
  GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}]]
```

In[*]:= term = T[la, lb] × A[ub, uc] × S[lc, uj] ×
T[ld, lf] × v[ud] × v[uf] × v[ua] × RiemannCD[lj, lk, ll, lm]

Out[*]=

$$A^{ab} R_{cklm} S_b^c T_{da} T_{ef} v^d v^e v^f$$

In[*]:= FindFreeTensorialIndices[term]

Out[*]=

{lk, ll, lm}

In[*]:= SP`xSort[term]

Out[*]=

```
xTensorTimes[
  xObject[Abc, {A}, {{ub, uc}, {ub, uc}, {}, {{1, Metricg, 1}, {1, Metricg, 1}}}],
  xObject[Rjklm, {RiemannCD}, {{lj, lk, ll, lm}, {lj, lk, ll, lm}, {},
    {{-1, Metricg, 1}, {-1, Metricg, 1}, {-1, Metricg, 1}, {-1, Metricg, 1}}}],
  xObject[Scj, {S}, {{lc, uj}, {lc, uj}, {},
    {{-1, Metricg, 1}, {1, Metricg, 1}}}], xCommutingObjects[xObject[Tab, {T},
    {{la, lb}, {la, lb}, {}, {{-1, Metricg, 1}, {-1, Metricg, 1}}}], xObject[
  Tdf, {T}, {{ld, lf}, {ld, lf}, {}, {{-1, Metricg, 1}, {-1, Metricg, 1}}}],
  xCommutingObjects[xObject[va, {v}, {{ua}, {ua}, {}, {{1, Metricg, 1}}}],
  xObject[vd, {v}, {{ud}, {ud}, {}, {{1, Metricg, 1}}}],
  xObject[vf, {v}, {{uf}, {uf}, {}, {{1, Metricg, 1}}]]]
```

In[*]:= SP`xSymmetryOf[term]

Out[*]=

```
xSymmetry[15, xTTimes[A12, R3456, S78, T910, T1112, v13, v14, v15],
  {1 → ub, 2 → uc, 3 → lj, 4 → lk, 5 → ll, 6 → lm, 7 → lc, 8 → uj,
  9 → la, 10 → lb, 11 → ld, 12 → lf, 13 → ua, 14 → ud, 15 → uf},
  GenSet[{Cycles[{{1, 2}}], -1}, {Cycles[{{3, 4}}], -1},
  {Cycles[{{5, 6}}], -1}, {Cycles[{{3, 5}, {4, 6}}], 1},
  {Cycles[{{7, 8}}], 1}, {Cycles[{{9, 11}, {10, 12}}], 1},
  {Cycles[{{13, 14}}], 1}, {Cycles[{{14, 15}}], 1}]]
```

In[*]:= A[uc, um] × RiemannCD[lj, lm, ll, ld] × B[la, lb] × T[lc, lj] × v[ua] × v[ub]
SP`xSymmetryOf[%]

Out[*]=

$A^{ab} B_{ce} R_{jbl d} T_{aj} v^c v^e$

Out[*]=

```
xSymmetry[12, xTTimes[A12, B34, R5678, T910, v11, v12],
  {1 → ua, 2 → ub, 3 → lc, 4 → le, 5 → lj, 6 → lb, 7 → ll,
  8 → ld, 9 → la, 10 → lj, 11 → uc, 12 → ue}, GenSet[
  {Cycles[{{1, 2}}], -1}, {Cycles[{{5, 6}}], -1}, {Cycles[{{7, 8}}], -1},
  {Cycles[{{5, 7}, {6, 8}}], 1}, {Cycles[{{11, 12}}], 1}]]
```